



HACK THE BOX · LINUX

Cap

Complete technical writeup ·
reproducible exploitation

Cap Machine Solved

HTB Cap · 2026-04-15

Author	Ramon Santos Sabarich / r4ms4nt
Type	Complete and archivable technical report
Objective	Document enumeration, exploitation, user and root with clear and reusable traceability
Resolution date	2026-04-15



Standardized technical report for
archiving, reference, and traceability.

READING GUIDE

Index

Consolidated technical report for reading, archiving, and reference. The content preserves the real sequence of enumeration, exploitation, pivot, privilege escalation, and lab closure.

- Environment preparation
- Initial reconnaissance
- Web enumeration and phase 1 closure
- Discovery of the useful PCAP resource
- Analysis of 0.pcap and credential extraction
- SSH access as nathan
- Obtaining user.txt
- Local enumeration for privilege escalation
- Key finding: capabilities on Python
- Escalation to root
- Obtaining root.txt
- Final technical summary



1. Environment preparation

Creating the working directory

Purpose of this step: create the case workspace and keep the content, exploits, and nmap folders separated.

Connecting to the HTB VPN

Purpose of this step: connect to the Hack The Box VPN in order to interact with the target machine.

Setting the target IP

Purpose of this step: make the target IP and machine name visible in the working environment.

2. Initial reconnaissance

Connectivity check

Purpose of this step: verify that the target machine is reachable over the network.

Quick system identification

Observed result:

Full port scan

Observed open ports:

- 21/tcp → FTP

- 22/tcp → SSH
- 80/tcp → HTTP

Relevant preserved output:

Service scan

Key findings:

- 21/tcp → vsftpd 3.0.3
- 22/tcp → OpenSSH 8.2p1 Ubuntu 4ubuntu0.2
- 80/tcp → gunicorn
- observed web title: Security Dashboard

Useful interpretation: the dominant surface becomes the web application, because port 80 is not just open; it exposes a named application with an identifiable backend.

3. Web enumeration and phase 1 closure

After accessing `http://10.129.28.170/`, the application shows a security dashboard and a visual session or context associated with Nathan is observed.

At that point, the reasonable phase 1 closure is:

- dominant surface: web
- main branch: WEB-BASE
- secondary branches: FTP, SSH
- single next step: profile the dashboard's real functionality and review resources exposed by the application

This decision is justified because:

- the web service returns a real application, not an empty landing page;
 - FTP and SSH do not yet provide useful access by themselves;
 - no valid credential or reusable login had yet been confirmed.
-
-

4. Discovery of the useful PCAP resource

Inside the dashboard, the following feature is accessed:

Navigation leads to a numbered route:

Important observation

The URL uses a numeric identifier, and a capture download button is also visible. This suggests that the application allows access to different PCAP files depending on the ID.

Test performed

- download of 1.pcap
- manual URL change to `http://10.129.28.170/data/0`
- download of 0.pcap

Observed result

- 1.pcap is empty
- 0.pcap contains useful traffic

Conclusion: the numeric ID is relevant, and the useful capture for the case is 0.pcap.

5. Analysis of 0.pcap and credential extraction

The file 0.pcap is opened with Wireshark.

Key finding

Inside the capture, a clear-text FTP session is observed with these credentials:

Validity confirmation

The capture itself shows:

Therefore, the credential is validated at least for FTP.

Other commands visible in the capture

Relevant observation

The RETR notes.txt operation ends with an error:

Therefore, the real value of the PCAP is not in that file, but in the reusable credential exposed in clear text.

Correct branch change

At this point, the main path stops being only web and becomes:

- dominant finding: valid credential
- new main branch: CREDENTIALS
- single next step: test reuse over SSH

6. SSH access as nathan

Connection

Password used:

Observed result

Successful access to the machine as nathan:

Initial checks

Results:

- effective user: nathan
 - UID/GID: 1001
 - user.txt visible in the user's home directory
-

7. Obtaining user.txt

Reading the user flag

User flag:

Terminal adjustment for more comfortable work

This adjustment is not part of the exploitation chain, but it is part of the operational work during the SSH session.

8. Local enumeration for privilege escalation

Searching for SUID binaries

Observed result: typical system SUID binaries appear, along with entries under snap, but in the preserved material no validated SUID exploitation path is derived from this search.

Searching for capabilities

Observed result:

Specific verification of the interesting binary

Result:

Useful interpretation: `/usr/bin/python3.8` can invoke `setuid(0)` thanks to its capability, which opens a direct escalation path.

9. Key finding: capabilities on Python

Before running the escalation, the current context is confirmed:

Result:

Then Python is used to elevate the effective UID to 0 and launch a shell:

```
python3 -c 'os.setuid(0); os.system("/bin/sh")'
```

10. Escalation to root

Confirmation of the new context

Observed result:

Technical note

The obtained shell correctly elevates to `uid=0`, although it preserves `gid=1001(nathan)` in the displayed output. For this lab, this is sufficient, since it allows reading the root flag and operating as the superuser.

Additional checks

Important observation: at first, `cat root.txt` fails because the shell is still located in nathan's home directory, not in `/root`.

The search confirms the correct location:

11. Obtaining root.txt

Final flag read

Root flag:

With this, the CAP machine is solved.

12. Final technical summary

Complete validated chain

1. Environment preparation and connection to HTB
2. Initial reconnaissance
3. Discovery of three services: FTP, SSH, HTTP
4. Identification of the web application as the initial dominant surface (Security Dashboard)
5. Access to Security Snapshot (5 Second PCAP + Analysis)
6. Discovery of the useful capture in `0.pcap`
7. Analysis of `0.pcap` with Wireshark
8. Extraction of clear-text FTP credentials
9. Reuse of nathan / Buck3tH4TF0RM3! over SSH
10. Obtaining `user.txt`
11. Local enumeration
12. Discovery of `cap_setuid` on `/usr/bin/python3.8`
13. Escalation to root through `os.setuid(0)`
14. Obtaining `root.txt`

Flags

- `user.txt` → `d318e8d5e5ebf7aa4f9b3b2f7c33aa16`
- `root.txt` → `420344e88a4a378d796944d13d7a0851`

Core idea of the case

CAP is a short but very didactic lab because it chains three clean ideas:

- a web exposure that leads to PCAP captures;
- a reusable credential visible in unencrypted network traffic;
- a simple local escalation based on Linux capabilities.

